```
  <body>
    <%= yield %>
  </body>
</html>
```

Let's add a second scenario to our PageViews system spec to test this behavior (remember, our aim here is to demonstrate how to configure JavaScript testing, so I'll trust you to write more useful tests in your own apps):

```
Line 1  require 'rails_helper'
-
-   RSpec.describe "PageViews" do
-     it "shows the number of page views" do
5       visit '/welcome'
-       expect(page.text).to match(/This page has been viewed [0-9]+ times?!/)
-     end
-
-     it "is enhanced with JavaScript on", js: true do
10      visit '/welcome'
-       expect(page).to have_text("ENHANCED!")
-     end
-   end
```

As per the RSpec convention, we've indicated that this new scenario (line 9) is only expected to pass with JavaScript enabled, by specifying js: true.

However, we have a problem. As you may know, the default driver used by Capybara is RackTest, which, although fast to run, doesn't have JavaScript support. If we were to run the system specs now, they would fail even though (we believe) this feature is working.

To be able to run specs that rely on JavaScript being executed in the application, we have to use a different, more full-fledged driver. There are a number of options:

- Selenium,[9] which supports several browsers including Chrome with its recently announced headless Chrome support[10]
- Capybara-webkit,[11] a driver for the headless WebKit implementation from the Qt cross-platform toolkit
- Poltergeist,[12] a driver for headless WebKit using PhantomJS[13]

9. https://github.com/teamcapybara/capybara#selenium
10. https://developers.google.com/web/updates/2017/04/headless-chrome
11. https://github.com/thoughtbot/capybara-webkit
12. https://github.com/teampoltergeist/poltergeist
13. http://phantomjs.org